API QA Report

REST API Testing with Postman

Project status: COMPLETED

System under test: JSONPlaceholder public mock REST API

Tools: Postman, JavaScript assertions, JSON

Test coverage: 9 planned scenarios across GET, POST, PUT, PATCH and DELETE

Report date: 25 October 2026

FINAL STATUS: All 9 test scenarios passed, as confirmed by the project owner.

This final report reflects the user's confirmation that all test cases passed. Detailed per-request assertion counts and execution timestamps were not provided for every scenario.

1. Executive Summary

The API Testing Portfolio project tested representative REST API operations against JSONPlaceholder using Postman. The test suite covered retrieval, creation, full update, partial update, deletion, response validation, and selected negative scenarios. The project owner confirmed that all test cases passed.

Automated post-response assertions were used to validate HTTP status codes, JSON response structure, required fields, data types, and selected response values. The project demonstrates a repeatable approach to functional API testing and result documentation.

2. Objectives and Scope

- Validate HTTP status codes for representative REST operations.
- Check JSON response format and expected response fields.
- Validate data types and selected field values.
- Exercise positive scenarios and selected negative scenarios.
- Use reusable Postman JavaScript assertions.
- Document observed behavior and the limitations of a public mock API.

3. Test Environment

API base URL	https://jsonplaceholder.typicode.com
API type	Public mock REST API
Test client	Postman
Assertion language	JavaScript (pm.test / pm.expect)
Response format	JSON
Environment variable	base_url

4. Test Coverage and Final Results

The project owner confirmed that all nine planned scenarios passed. The table below summarizes the covered scenarios.

Test Case ID	Scenario	Method	Endpoint	Final Status
TC-API-001	Fetch a single post	GET	/posts/1	**PASS**
TC-API-002	Fetch all posts	GET	/posts	**PASS**
TC-API-003	Fetch nonexistent post	GET	/posts/999999	**PASS**
TC-API-004	Create a post	POST	/posts	**PASS**
TC-API-005	Create post without title	POST	/posts	**PASS**
TC-API-006	Update a post	PUT	/posts/1	**PASS**
TC-API-007	Partially update a post	PATCH	/posts/1	**PASS**
TC-API-008	Delete an existing post	DELETE	/posts/1	**PASS**
TC-API-009	Delete nonexistent post	DELETE	/posts/999999	**PASS**

5. Validation Areas

- **Status-code validation:** Assertions checked expected HTTP status codes for each scenario.
- **Response format:** Responses were checked for valid JSON where appropriate.
- **Response structure and fields:** Assertions checked required properties and selected values, including IDs and submitted fields.
- **Data types:** Checks included numeric IDs and string fields where applicable.
- **Negative scenarios:** Requests included a nonexistent GET resource and a POST payload without a title, as well as deletion of a nonexistent resource.

6. Findings and Interpretation

- **Nonexistent GET:** The request for /posts/999999 returned HTTP 404. The response body was observed as an empty JSON object ({}), so the assertion should validate the actual response shape rather than assume an empty string.
- **Missing-title POST:** JSONPlaceholder returned HTTP 201 and omitted the title field for the request without a title. This is recorded as the mock API's behavior; whether this is acceptable in a production API depends on its contract.
- **DELETE behavior:** The existing and nonexistent DELETE requests returned HTTP 200 in the observed runs. For this mock API, this behavior is documented rather than generalized to production systems.

7. Limitations

- JSONPlaceholder is a mock API. POST, PUT, PATCH and DELETE operations are simulated; changes are not permanently persisted.
- Expected outcomes for production APIs should be based on documented API contracts and business requirements.
- Response-time assertions used during practice are demonstration thresholds, not formal performance SLAs.
- The overall pass status is based on the project owner's confirmation. Individual execution timestamps and per-request assertion counts were not supplied for every scenario.

8. Conclusion

The project is marked completed, with all nine planned test scenarios reported as passed by the project owner. It demonstrates practical Postman-based API testing, reusable assertions, response validation, and negative-scenario coverage. The collection, test-case workbook, screenshots, README, and this report can be presented together as a freelance QA portfolio, with the mock API limitations clearly disclosed.